

Lab0 实验报告

1. 我此前没有多人协同开发的经历，主要独立完成编程作业。通过本次实验，我了解到团队可以按功能分工，每个人在独立分支上开发，完成后再合并，并共同处理冲突。

暂存区让开发者选择本次提交包含哪些修改，提交则将暂存区中的内容记录为一个版本。

例如，我同时修改了程序代码和实验报告，可以先只暂存并提交代码，再单独提交报告。这样，每次提交都有明确的目的，方便查看历史、审查修改和恢复版本。暂存区也允许只选取一个文件中的部分修改，使提交更加灵活。

不带参数的 `git branch` 列出本地分支，并用星号 `*` 标记当前所在的分支。

`git branch -a` 同时列出本地分支和远程跟踪分支，例如 `remotes/origin/main`。远程跟踪分支反映的是本地记录的远程状态，并非实时查询服务器，可以通过 `git fetch` 更新。

2. 已提交

3.我认为，学习 **Git** 不仅是为了保存代码，更是为了有条理地管理开发过程和开展团队协作。

阅读 **Git Flow** 后，我了解到，不同分支可以承担功能开发、版本整合和发布准备等任务。团队成员能够在各自的功能分支上工作，再将成果合并，从而减少相互干扰。阅读 **Commit Message** 规范后，我认识到，清晰的提交说明能够记录每次修改的目的，方便其他人理解代码变化，也便于日后查找问题和维护项目。

在本次实验中，我实际进行了提交、分支切换和冲突处理，体会到 **Git** 能够保留修改历史，并帮助我们整合不同分支的代码。因此，学习 **Git** 既能提高个人开发效率，也能培养规范记录、合理分工和协作解决问题的习惯。

4.

C main.c ! X

C main.c

```
1 #include <stdio.h>
2
3 int main()
4 {
5     // @TODO: print a sentence you want.
6     采用当前更改 | 采用传入的更改 | 保留双方更改 | 比较变更
7     <<<<<< HEAD (当前更改)
8     printf("Hello from main!\n");
9     =====
10    printf("Hello from feature!\n");
11    >>>>>> feature (传入的更改)
12 }
```

在合并编辑器中解决

问题 输出 调试控制台 终端 端口

bash - GitLab

```
xr@LAPTOP-1C8MDIMM:/mnt/c/Users/xr/GitLab$ git switch main
xr@LAPTOP-1C8MDIMM:/mnt/c/Users/xr/GitLab$ git add main.c
git commit -m "feat: change greeting on main"
git merge feature
[main c47c776] feat: change greeting on main
1 file changed, 1 insertion(+), 1 deletion(-)
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
xr@LAPTOP-1C8MDIMM:/mnt/c/Users/xr/GitLab$
```

C main.c ! X

C main.c

```
1 #include <stdio.h>
2
3 int main()
4 {
5     // @TODO: print a sentence you want.
6     printf("Hello from main and feature!\n");
7 }
```

问题 输出 调试控制台 终端 端口

git - GitLab

```
* 47ce017 (HEAD -> main) merge: resolve conflict between main and feature
|\
| * f419399 (feature) feat: change greeting on feature
* | c47c776 feat: change greeting on main
|/
* 9bd3734 (origin/main, origin/HEAD) merge: resolve main.c conflict
|\
| * 88223b2 Initial commit
* 4fd8766 Complete TODO in main.c
* f67e080 feat(): add autograding; doc(): add README.md
```